Chapitre 2

Communication dans les applications réparties

Définition de la communication répartie

Dans un **système réparti**, les composants logiciels et matériels sont situés sur des machines différentes reliées par un réseau. Pour coopérer, ces composants doivent échanger des informations : c'est le rôle de la communication répartie.

La **communication répartie** désigne l'ensemble des mécanismes, protocoles et modèles permettant à deux entités d'un système distribué (processus, objets, applications) d'échanger des données et de coordonner leurs actions, malgré la séparation physique des machines.

Elle peut se faire sous différentes formes : transmission de messages, appels de procédures distantes, invocation de méthodes sur des objets distants, ou encore échanges via des middlewares spécialisés.

➡ **Objectif :** donner l'illusion que les interactions à distance se déroulent aussi simplement qu'en local, tout en masquant la complexité du réseau.

Importance et rôle de la communication

La communication constitue la **colonne vertébrale des applications réparties**. Elle permet :

La coopération entre composants : par exemple, un client envoie une requête à un serveur pour accéder à une base de données distante.

Le partage de ressources : fichiers, services web, systèmes d'information.

La coordination des tâches : plusieurs processus répartis sur différentes machines peuvent travailler ensemble pour résoudre un problème complexe.

La transparence : grâce aux mécanismes de communication, l'utilisateur n'a pas besoin de savoir où se trouvent réellement les ressources ou comment elles sont accessibles.

➡ Sans communication fiable, les applications réparties perdraient leur sens : elles ne seraient qu'un ensemble de composants isolés, incapables d'interagir.

Défis de la communication répartie

1 Hétérogénéité :

Les nœuds d'un système réparti peuvent être très différents : PC, smartphones, serveurs, objets IoT… avec des systèmes d'exploitation, langages et protocoles variés.

Défi : assurer une communication interopérable malgré ces différences.

2 Transparence de la localisation :

Dans un système réparti, un composant ne doit pas avoir à savoir *où* se trouve l'autre composant (communication qui ressemble à un appel local alors que le serveur est à l'autre bout du monde).

Défi : masquer la localisation réelle (IP, port, machine) derrière une abstraction.

3 Latence et performance du réseau :

Les messages circulent sur des réseaux avec une certaine latence, vitesse de transmission limitée et congestion possible (Exemple : 200ms de retard dans une visioconférence sont perceptibles).

Défi : assurer une communication efficace malgré des délais.

Défis de la communication répartie

4 Fiabilité et pannes :

Les réseaux sont peu fiables : messages perdus, dupliqués, corrompus, ou nœuds qui tombent en panne (Exemple : en cas de panne réseau, il faut éviter qu'un virement soit débité deux fois).
Défi : garantir la cohérence et la tolérance aux fautes et concevoir des mécanismes de reprise, de réplication et de redondance.

5 Concurrence et synchronisation :

Plusieurs nœuds peuvent accéder en même temps à la même ressource (Exemple : deux utilisateurs modifient en même temps un même document collaboratif).

Défi : gérer les accès concurrents, éviter les conflits et maintenir la cohérence.

6 Sécurité :

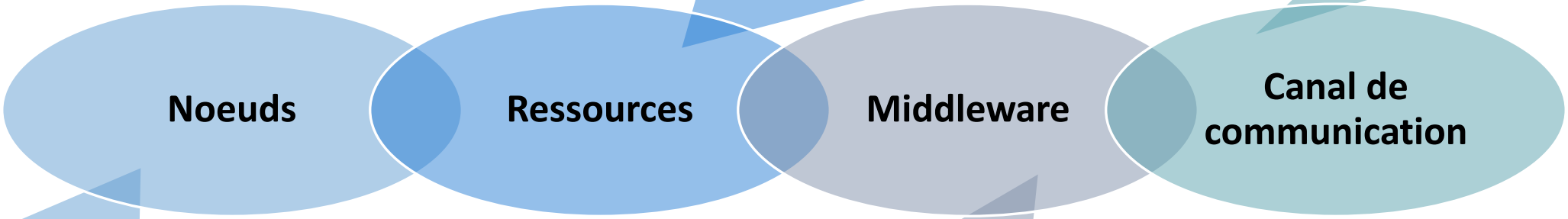
La communication sur un réseau ouvert (comme Internet) peut être interceptée, modifiée ou usurpée.

Défi : assurer authentification, confidentialité, intégrité et autorisation.

Les intervenants de la communication répartie

Ce sont les éléments accessibles ou partagés dans le système (fichiers, bases de données, services, périphériques, puissance de calcul…).
→ C'est le contenu demandé/fourni lors de la communication.

C'est le support de transmission des données (réseau filaire, Wi-Fi, Internet, protocoles de communication).

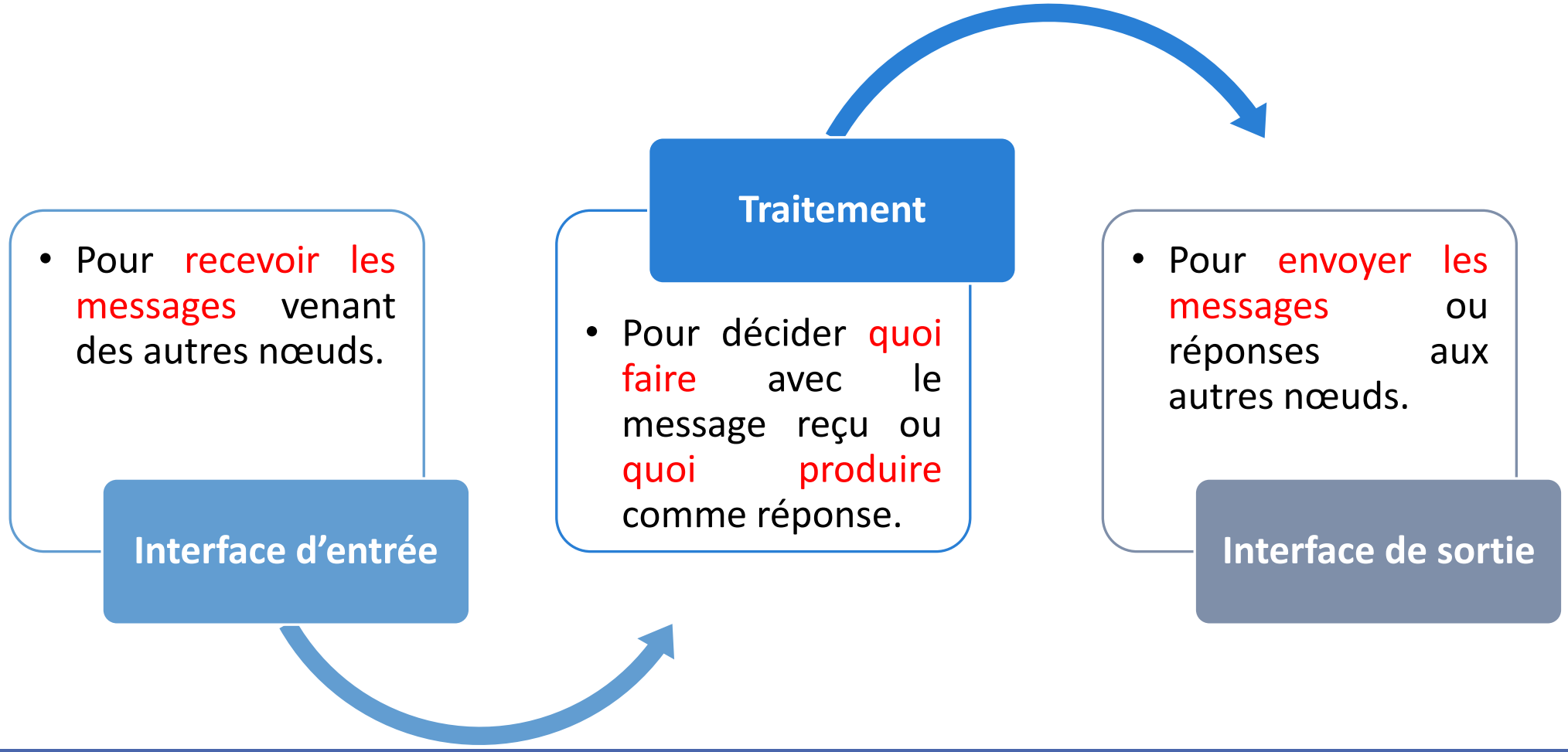


Ce sont les entités participant au système réparti. Ils peuvent correspondre soit à des machines physiques (ordinateurs, serveurs, smartphones, capteurs IoT, etc.), soit à des unités logiques (processus, services ou applications)

C'est la couche logicielle intermédiaire qui facilite la communication et la transparence entre nœuds (CORBA, Java RMI, …).

Composition logique d'un noeud

Dans un système réparti, un nœud, quelque soit son type, correspond à un bloc logique avec trois fonctions : recevoir, traiter, envoyer des informations.



Les modes de communication

Les modes définissent le comportement temporel de l'échange (temps réel ou différé). Ils peuvent s'appliquer à plusieurs modèles de communication.

Communication synchrone

Dans ce mode, l'émetteur envoie une requête et attend la réponse avant de poursuivre son exécution.

→ La communication se fait en temps réel entre les deux parties, avec un blocage possible en attente de la réponse.

Avantage : simplicité et rapidité de l'échange direct.
Limite : dépendance temporelle forte, le client est bloqué tant que la réponse n'arrive pas.

Communication asynchrone

L'émetteur envoie un message sans attendre immédiatement la réponse. Le receveur peut répondre plus tard.

→ Cela permet à l'émetteur de continuer son traitement sans blocage.

Avantage : meilleure tolérance aux délais et aux interruptions.
Limite : complexité plus élevée pour gérer les retours différés.

Les mécanismes d'adressage et de diffusion

Communication Unicast

- communication entre un seul émetteur et un seul récepteur (un à un).
- **Exemple :** une requête d'un client vers un serveur.

Communication multicast

- un émetteur envoie un message à un groupe spécifique de récepteurs.
- **Exemple :** une visioconférence est diffusée uniquement aux participants.

Communication broadcast

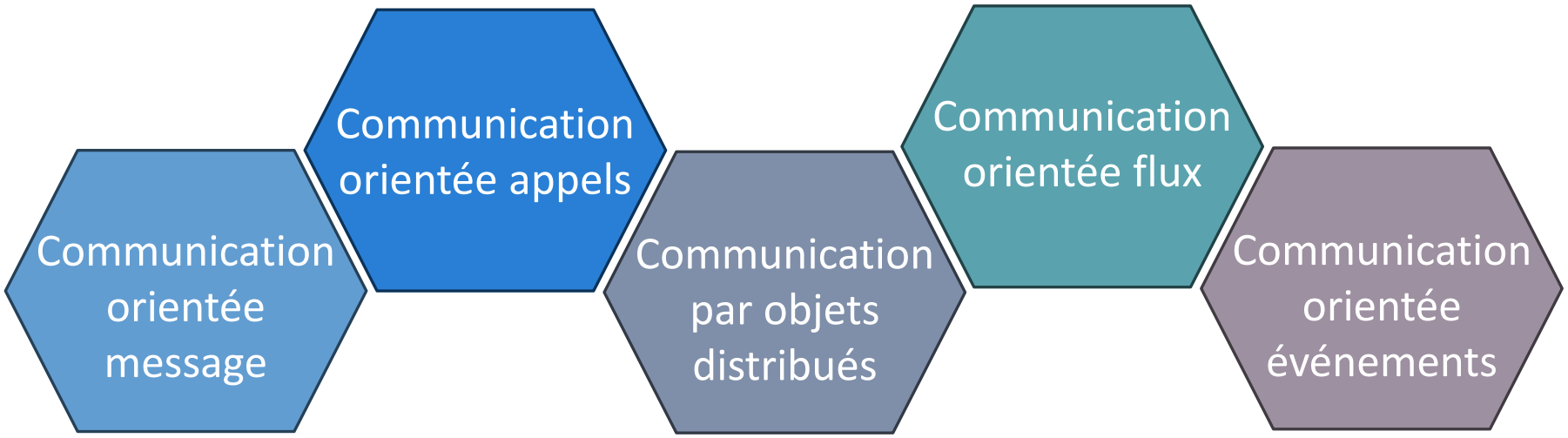
- un émetteur diffuse le message à tous les nœuds d'un réseau.
- **Exemple :** un message envoyé simultanément à tous les abonnés du réseau mobile.

Les modèles de communication

La communication dans un système réparti peut prendre plusieurs formes, selon la nature de l'échange et le degré de synchronisation attendu entre les entités.

Les modèles de communication décrivent la structure, la dynamique ou la forme globale du processus de communication. Ils précisent comment l'information est transmise entre les acteurs.

On distingue principalement :



Communication orientée message

Principe

Les nœuds (ou les entités) échangent des informations sous forme de messages explicites (envoyer / recevoir).

Caractéristiques :

- ✓ Les messages peuvent être synchrones ou asynchrones.
- ✓ La communication se fait par des files de messages, où un message est envoyé par un producteur et consommé par un consommateur.
- ✓ Nécessite une adresse de destination.
- ✓ Implémenté via des **sockets** réseau (**TCP ou UDP**)

Avantages

Découplage fort (les nœuds peuvent être écrits dans des langages différents, être sur des machines distantes, etc.)

Inconvénients

Complexité de gestion des messages (ordre, duplication, erreurs…)

Exemple concret

- ➢ Une application bancaire envoie une requête de virement sous forme de message à un service de paiement. Le service reçoit, traite, et renvoie une confirmation par message.

Communication orientée appels

Principe

La communication par appel de procédure à distance **(RPC - Remote Procedure Call)** consiste à appeler à distance une procédure ou une fonction sur un autre système comme si elle était locale.

Caractéristiques :

- ✓ Le système cache la complexité réseau via des stubs (proxy local) qui transforment les appels en messages et les réponses en résultats.
- ✓ La communication est généralement synchrone
- ✓ Le programmeur n'a pas à gérer l'envoi/réception de messages.

Avantages

Simplicité d'utilisation (on programme comme si tout était local)

Inconvénients

- Forte dépendance au réseau (défaillances, latence, …)
- Couplage fort : Chaque nœud doit connaître la signature de la fonction distante (nom, paramètres, types de retour)

Exemple concret

Une application mobile appelle payerFacture() sur un serveur distant via RPC. Pour le développeur, cela ressemble à un simple appel de fonction.

Communication par objets distribués

Principe

Dans la communication par objets distribués **(RMI - Remote Method Invocation)**, les entités du système réparti sont représentées par des objets distants. Chaque objet expose ses fonctionnalités au travers d'une interface distante, et les autres entités peuvent invoquer ses méthodes de manière transparente comme s'il s'agissait d'objets locaux.

Caractéristiques :

- ✓ C'est une extension de l'idée du RPC, mais appliquée aux objets.
- ✓ Un objet sur une machine peut invoquer une méthode sur un objet situé ailleurs.
- ✓ Implémenté par des middlewares comme **Java RMI** et **CORBA**.

Avantages

Transparence de localisation; modularité et réutilisation d'objets; orientation objet.

Inconvénients

Forte dépendance technologique, problème de compatibilité de versions

Exemple concret

Dans un système de gestion d'hôtel réparti, l'objet Réservation appelle la méthode confirmer() sur l'objet Chambre, qui réside sur un autre nœud du système.

Communication orientée flux

Principe

La transmission orientée flux est un modèle de communication en temps réel basé sur le transfert continu de données entre producteurs et consommateurs.

Caractéristiques :

- ✓ Les données sont envoyées sous forme de flux (audio, vidéo).
- ✓ Peut être unidirectionnel (ex : diffusion vidéo) ou bidirectionnel (ex : visioconférence).

Avantages

Transmission continue et temps réel, ce qui le rend idéal pour les flux multimédias.

Inconvénients

Nécessite la gestion de la latence, la synchronisation et parfois la qualité de service (QoS).

Exemple concret

- Netflix ou YouTube utilisent un flux pour diffuser une vidéo à un utilisateur.
- Zoom / Teams utilisent des flux audio/vidéo bidirectionnels pour une réunion.

Communication orientée événements

Principe

Elle repose sur un modèle pub/sub (publication/abonnement) où les émetteurs (publishers) publient des événements sans connaître les récepteurs, tandis que les récepteurs (subscribers) s'abonnent à des types d'événements qui les intéressent.

Caractéristiques :

- ✓ Permet la diffusion à plusieurs récepteurs (multicast, broadcast).
- ✓ Asynchrone par nature, adapté aux systèmes réactifs et distribués.

Avantages

Favorise un couplage faible, une grande scalabilité, et une distribution flexible des notifications.

Inconvénients

- – Difficulté à suivre les flux de données
- – Complexité de gestion : Un même événement peut être reçu par plusieurs services. Chacun peut réagir de manière différente.

Exemple concret

Un client passe une commande sur un site de vente en ligne. L'action "commande confirmée" déclenche la publication d'un événement **commande/nouvelle**. Plusieurs services abonnés réagissent :

- ✓ Service de paiement : lance la procédure de transaction.
- ✓ Service logistique : prépare l'emballage et l'expédition.
- ✓ Service client : envoie un email de confirmation au client.
- ✓ Module de fidélité : ajoute des points au compte du client.

Le rôle du réseau dans la communication

Une application répartie est souvent déployée sur plusieurs machines situées dans des zones géographiques différentes. Ces composants doivent échanger des données à travers un réseau pour fonctionner comme un tout cohérent.

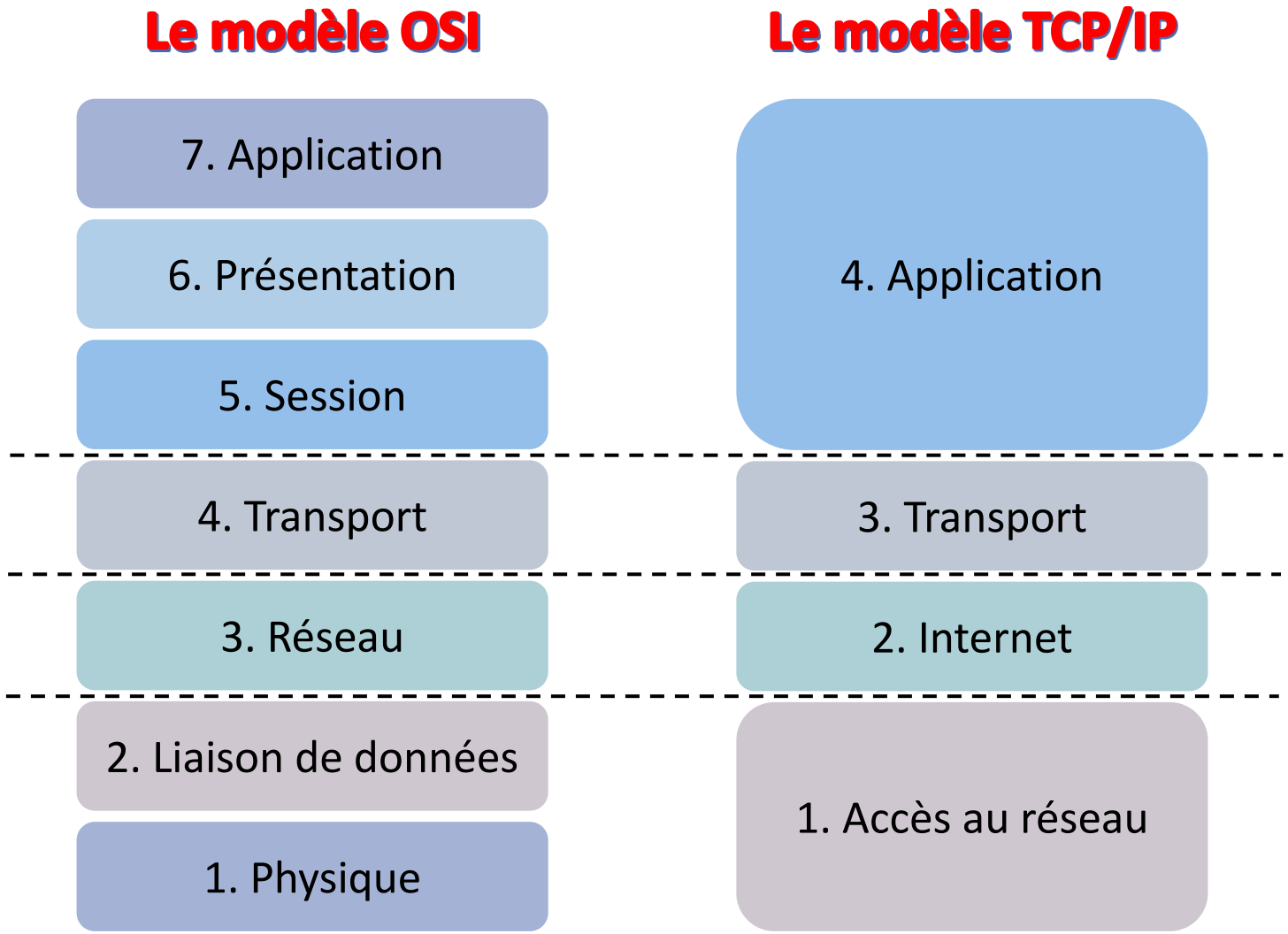
➡ Le réseau devient alors le canal de transport des informations entre les différents noeuds.

Pour que les données puissent circuler correctement entre machines, il faut une organisation logique du réseau. Cette organisation est décrite à l'aide de modèles en couches, qui indiquent comment les données passent d'un nœud à un autre.

Il existe deux modèles en couches de référence principaux :

- ✓ **Modèle OSI (Open Systems Interconnection) : Modèle théorique** défini à la fin des années 1970 par l'ISO (Organisation Internationale de Normalisation). Son but était de standardiser la communication entre ordinateurs, en définissant **7 couches** bien distinctes, chacune ayant un rôle précis.
- ✓ **Modèle Internet (TCP/IP : Transmission Control Protocol/Internet Protocol) : Modèle pratique** décrivant l'architecture réellement utilisée dans les réseaux actuels (Internet, intranets, etc.). Il simplifie les 7 couches de l'OSI en **4 grandes couches** fonctionnelles, plus faciles à gérer et à coder.

L’architecture en couches des réseaux



Le modèle OSI

Le modèle OSI

- **7. Application**
- 6. Présentation
- 5. Session
- 4. Transport
- 3. Réseau
- 2. Liaison de données
- 1. Physique

La couche « Application » :

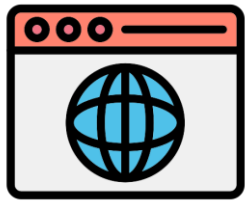
C'est l'interface entre l'utilisateur et le réseau. Elle permet à l'utilisateur d'accéder à des ressources à distance (page web, fichiers, mails...).

Les protocoles de communication de la couche d'application comprennent :

- ✓ Le SMTP (Simple Mail Transfer Protocol) pour l'envoi des mails d'un serveur à un autre.
- ✓ Le HTTP (HyperText Transfer Protocol), qui est le protocole de communication entre les navigateurs et les serveurs Web, basé sur une requête-réponse.
- ✓ Et le FTP (File Transfer Protocol) spécifiquement conçu pour le transfert de fichiers entre ordinateurs.



SMTP



HTTP



FTP

Le modèle OSI

Le modèle OSI

7. Application

6. Présentation

5. Session

4. Transport

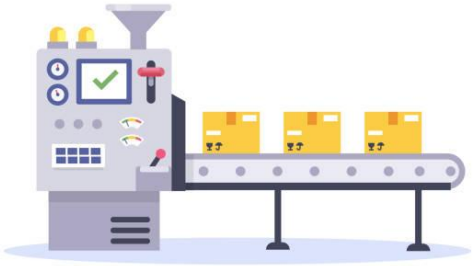
3. Réseau

2. Liaison de données

1. Physique

La couche « Présentation » :

Cette couche s'occupe du format des données échangées. Elle garantit que les données envoyées par une machine sont comprises correctement par une autre.



- ✓ Elle convertit les données d'un format interne à un format standard pour le réseau, puis inversement à la reception (conversion du texte d'un encodage UTF-8 vers ASCII, ou de JPEG vers BMP).
- ✓ Réduit la taille des données avant l'envoi (gain de bande passante) et les restaure à la réception (compression d'un fichier image ou d'un flux vidéo avant transfert).
- ✓ Protège la confidentialité des données échangées en les cryptant avant envoi, puis en les décryptant à la réception (HTTPS chiffre les données avant de les envoyer sur le réseau)..

Le modèle OSI

Le modèle OSI

7. Application
6. Présentation
5. Session
4. Transport
3. Réseau
2. Liaison de données
1. Physique

La couche « Session » :

Avant que deux applications puissent échanger des données, elles doivent établir une session de communication. Cette session agit comme un canal logique entre les deux appareils.

La couche session s'occupe :

- ✓ d'ouvrir la session au début de la communication,
- ✓ de la maintenir active pendant le transfert des données,
- ✓ puis de la fermer proprement une fois l'échange terminé.

Lorsqu'un grand volume de données est transmis, cette couche veille à la continuité de la session et à la reprise en cas d'interruption.



Le modèle OSI

Le modèle OSI

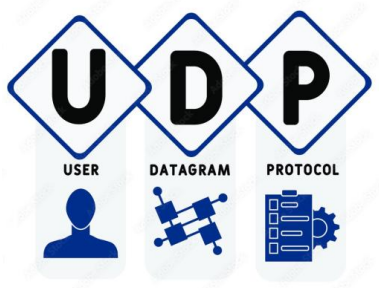
7. Application
6. Présentation
5. Session
4. Transport
3. Réseau
2. Liaison de données
1. Physique

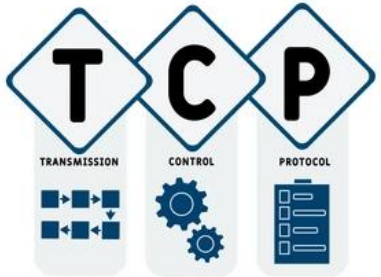
La couche « Transport » :

La couche transport a pour rôle d'assurer le transfert fiable et ordonné des données entre deux machines. Elle découpe les données en petits morceaux appelés segments, afin de faciliter leur envoi sur le réseau. Chaque segment contient un en-tête (numéro du segment, numéro du port, …) permettant de réassembler correctement le message à la réception.

Elle intègre aussi des mécanismes de contrôle d'erreur : si certains segments sont perdus ou endommagés, la couche transport demande à la couche « session » leur réémission, garantissant ainsi une communication complète et fiable.

La couche transport choisit la manière d'envoyer les informations selon le type de communication souhaitée :





Le modèle OSI

Le modèle OSI

7. Application
6. Présentation
5. Session
4. Transport
3. Réseau
2. Liaison de données
1. Physique

La couche « réseau » :

La couche réseau reçoit les segments de la couche transport et les encapsule dans des paquets IP, contenant l'adresse **IP** source et destination. Elle a pour rôle d'acheminer les paquets d'une machine à une autre, même si elles se trouvent sur des réseaux différents.

Si un paquet est trop volumineux pour le réseau (au-delà de la MTU – Maximum Transmission Unit), il est fragmenté en plusieurs parties plus petites, puis réassemblé à la réception.

Elle s'occupe aussi du routage, c'est-à-dire de trouver le meilleur chemin pour faire circuler les paquets entre plusieurs réseaux.



Le modèle OSI

Le modèle OSI

7. Application
6. Présentation
5. Session
4. Transport
3. Réseau
2. Liaison de données
1. Physique

La couche « liaison de données » :

La couche liaison de données reçoit les paquets de la couche réseau et les transforme en **trames**, prêtes à être envoyées physiquement.

Une trame → C'est le conteneur (suite de bits 0 et 1) qui circule directement sur le support physique (le câble, le Wi-Fi, etc.) entre deux machines reliées au même réseau local (LAN).

Elle assure le transfert fiable des données entre deux machines situées sur le même réseau local.

Elle utilise les adresses MAC pour identifier l'émetteur et le destinataire sur le réseau local, et veille à ce que les données soient bien reçues sans erreur.



Le modèle OSI

Le modèle OSI

7. Application
6. Présentation
5. Session
4. Transport
3. Réseau
2. Liaison de données
1. Physique

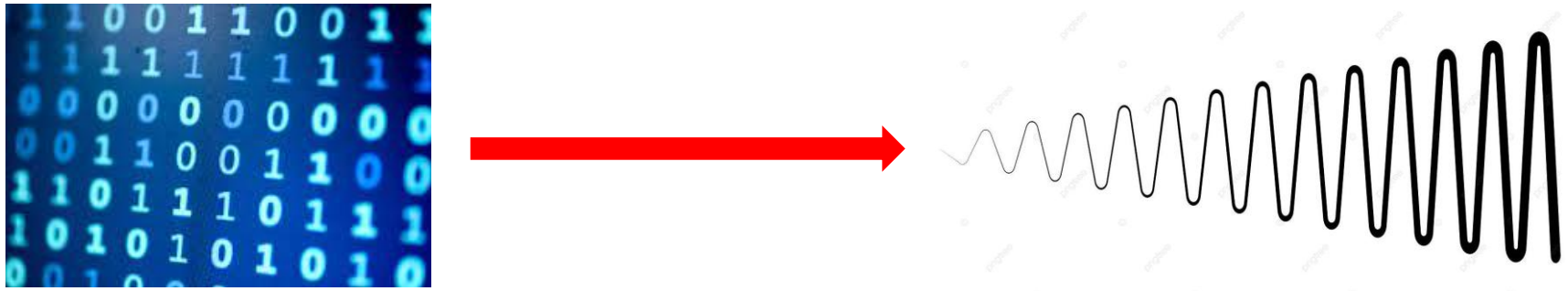
La couche « physique » :

Cette couche s'occupe de transmettre réellement les bits sur le support de communication.

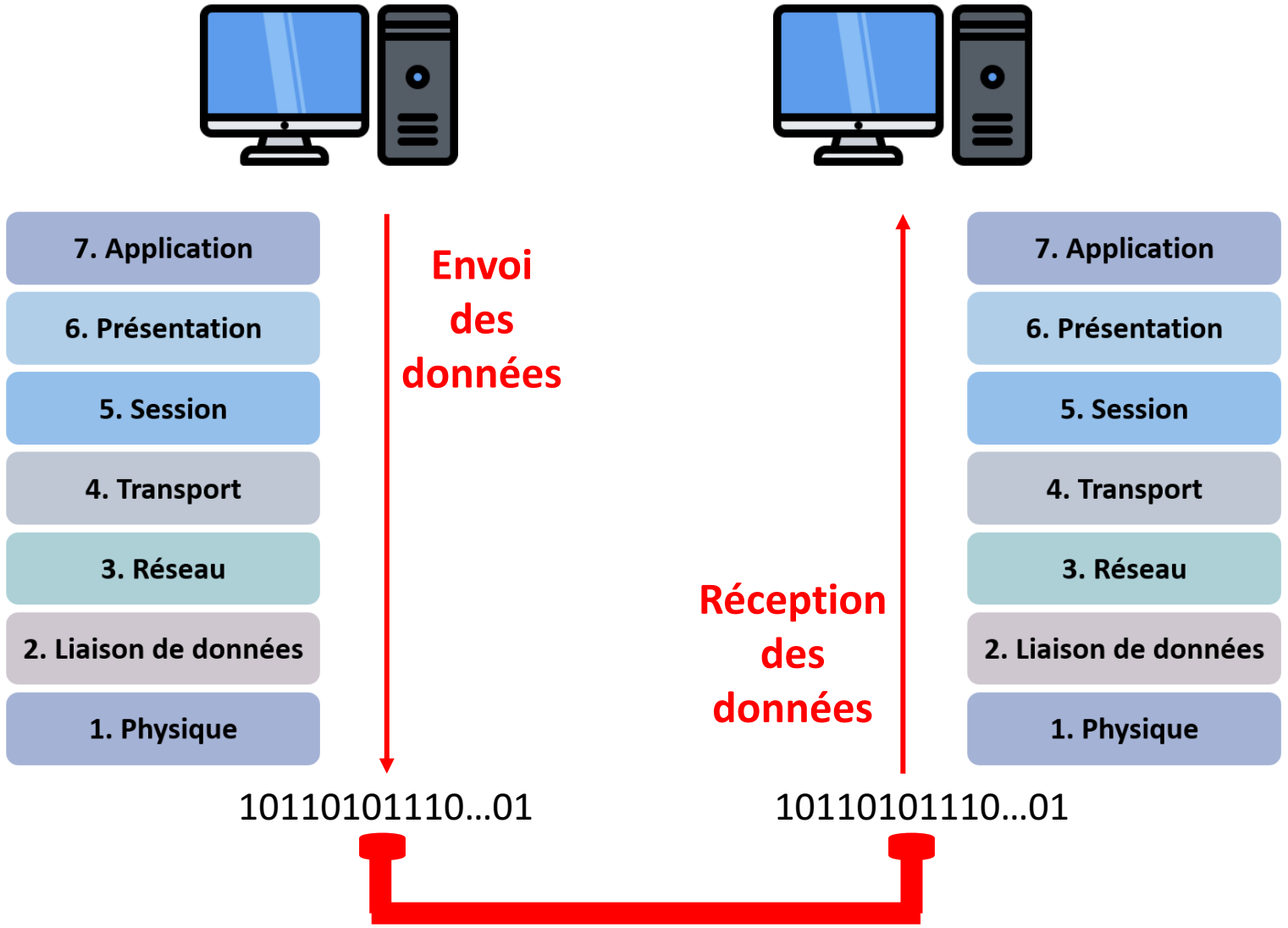
Elle convertit ces bits en signaux :

- signaux électriques si on utilise un câble en cuivre (Ethernet),
- signaux lumineux si on utilise une fibre optique,
- ondes radio si on utilise le Wi-Fi ou le Bluetooth.

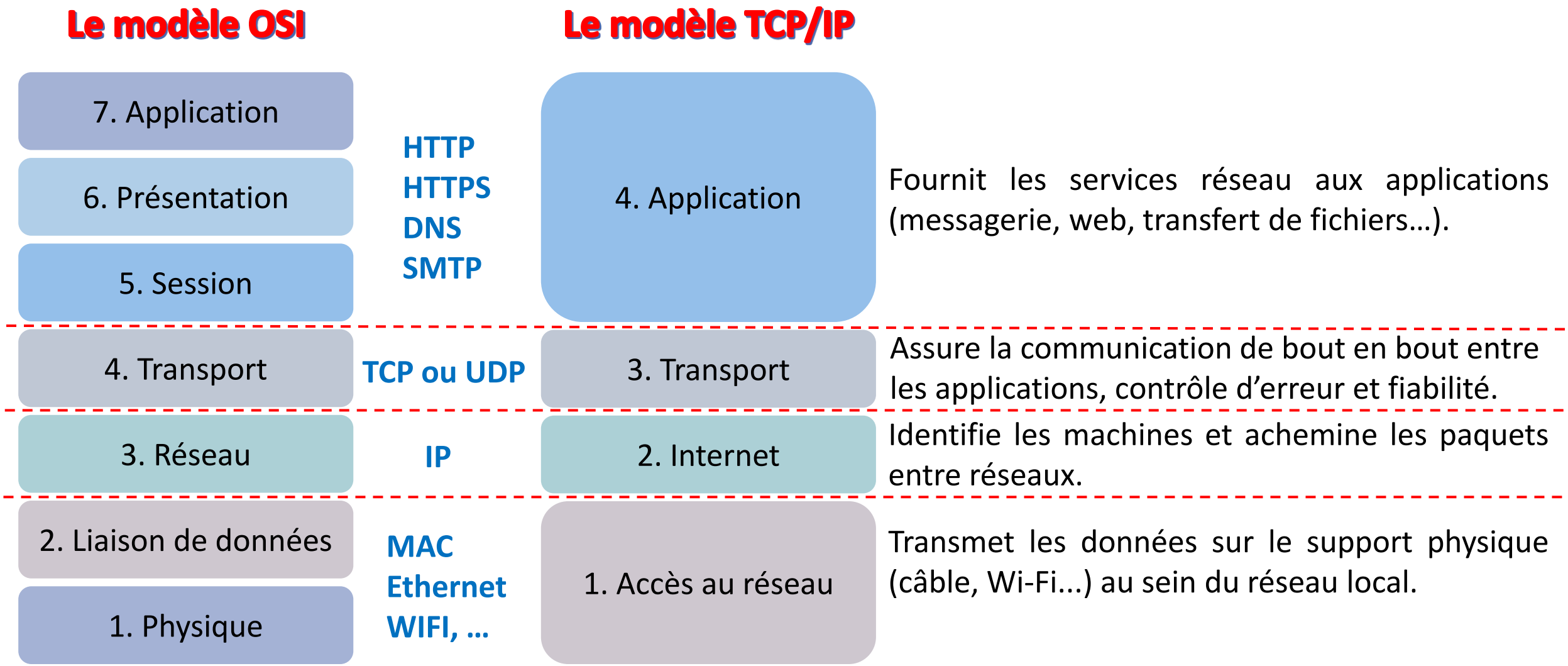
Elle envoie ces signaux sur le support de transmission (câble, fibre, ondes…).



Le modèle OSI



Le modèle TCP/IP (Internet)



Le modèle TCP/IP (Internet)

Chaque couche ajoute ses propres informations aux données reçues de la couche supérieure.





Chaque couche lit et retire ses en-têtes pour ne garder que les données destinées à la couche supérieure.

Encapsulation

Message

4. Application

Message + en-tête → Segment

3. Transport

Segment + IP → Paquet

2. Internet

Paquet + MAC → Trame → Bits

1. Accès au réseau

Encapsulation

Bits

10110101110…01

Désencapsulation

4. Application

Message

3. Transport

Segment

2. Internet

Paquet

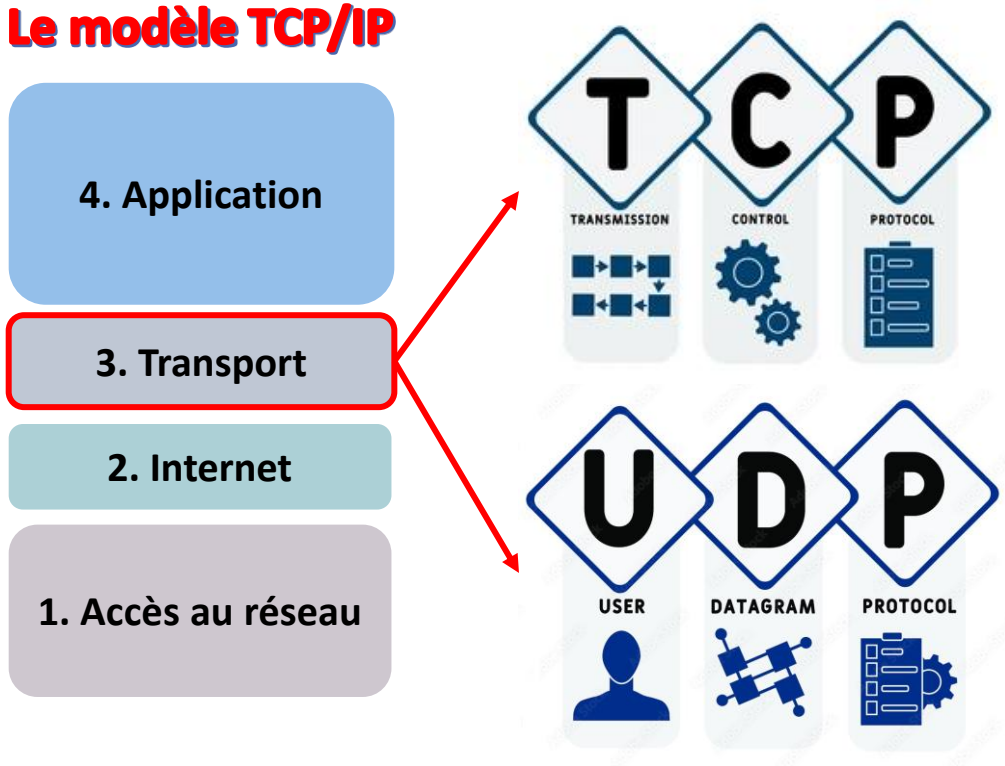
1. Accès au réseau

Trame

Désencapsulation

Les protocoles de transport des données

Le modèle TCP/IP



La couche Transport occupe une place centrale dans la communication réseau. Elle garantit un transfert de données fiable et ordonné entre les applications situées sur des machines distantes.

Elle découpe les données provenant de la couche Application en segments, leur ajoute des en-têtes puis supervise l'envoi de ces segments à travers le réseau.

Les deux principaux protocoles utilisés à ce niveau sont :

- ✓ **TCP (Transmission Control Protocol)**
- ✓ **UDP (User Datagram Protocol)**

Qu'est ce qu'un protocole ?

Un protocole en réseau informatique est un ensemble de règles et de normes qui définissent comment les nœuds communiquent entre eux.

Le protocole TCP (Transmission Control Protocol)

TCP est un protocole de transport orienté connexion, c'est-à-dire qu'avant tout échange de données, une connexion fiable est établie entre l'émetteur et le récepteur.

Il garantit que les données arrivent dans le bon ordre, sans perte, et sans duplication.

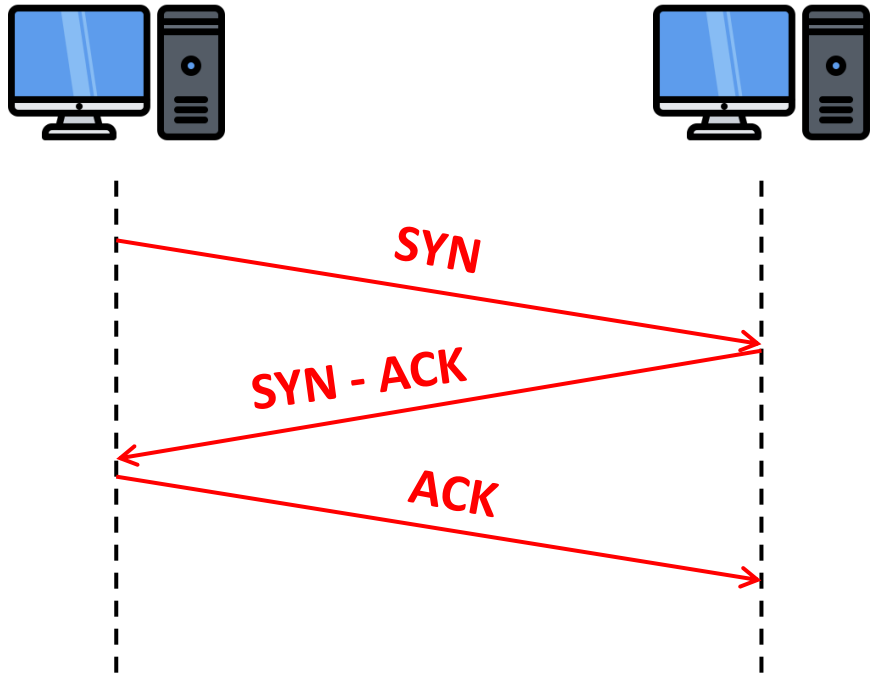
1 Etablissement de la connexion :

TCP utilise le mécanisme de handshake en trois étapes (3-way handshake) :

1. Le client envoie une demande de connexion (SYN).
2. Le serveur répond (SYN-ACK).
3. Le client confirme (ACK).

Une fois cette connexion établie, les deux parties peuvent échanger des données.

SYN : Synchronize (commencer la synchronization)
ACK : Acknowledgement (accusé de réception)



Le protocole TCP (Transmission Control Protocol)

2 Transfert des données :

- Les données sont découpées en segments.
- Chaque segment reçoit un numéro de séquence pour garantir l'ordre d'arrivée.
- Si un segment est perdu, il est retransmis.
- Les accusés de réception (ACK) permettent de confirmer la bonne réception des données.

3 Fermeture de la connexion :

Après la transmission, la connexion est fermée proprement pour libérer les ressources.

Avantages

- ✓ Transmission fiable
- ✓ Contrôle de flux et de congestion
- ✓ Réassemblage automatique des segments

Inconvénients

- × Lent à cause des mécanismes de contrôle et de vérification
- × Consomme plus de ressources réseau

Le protocole TCP (Transmission Control Protocol)

Cas d'utilisation courants du protocole TCP :

- ✓ **Navigation Web (HTTP/HTTPS)** : TCP garantit la transmission complète et sans erreur des pages, images et ressources web, assurant une navigation fluide et fiable.
- ✓ **Courrier électronique (SMTP, IMAP, POP3)** : Il veille à ce que les e-mails soient envoyés et reçus exactement comme ils ont été produits, sans perte ni altération.
- ✓ **Transfert de fichiers (FTP, SFTP)** : TCP assure une transmission fiable et intègre des fichiers lors des téléchargements ou des envois.
- ✓ **Accès sécurisé à distance (SSH)** : Les connexions SSH utilisent TCP pour garantir la fiabilité et l'intégrité des données échangées de manière chiffrée.
- ✓ **Accès à des bases de données distantes** : Les systèmes tels que MySQL ou PostgreSQL reposent sur TCP pour des transactions cohérentes et sûres entre client et serveur.
- ✓ **Applications de messagerie et de communication** : Des outils comme WhatsApp, iMessage ou Google Chat exploitent TCP afin d'assurer la livraison complète et ordonnée des messages textuels.

Le protocole UDP (User Datagram Protocol)

UDP est un protocole de transport non orienté connexion. Il envoie les données sous forme de datagrammes sans établir de connexion préalable et sans garantir leur livraison.

Fonctionnement

- Aucune phase de connexion : les datagrammes sont envoyés directement.
- Pas de numérotation ni d'accusé de réception.
- Si un datagramme est perdu ou arrive en désordre, UDP ne tente pas de le corriger.
- La vérification d'erreurs est minimale (calcul facultatif de la valeur checksum).

Un checksum (somme de contrôle) est une valeur calculée à partir du contenu d'un message.

- L'émetteur calcule une valeur de contrôle à partir des données du message.
- Il place cette valeur dans le champ checksum de l'en-tête UDP.
- Le récepteur recalcule le checksum à partir des données reçues.
- Si les deux valeurs ne correspondent pas → Détection d'une erreur.

Pas de retransmission : Si une erreur est détectée, le datagramme est simplement ignoré par le récepteur.

Le protocole UDP (User Datagram Protocol)

Avantages

- ✓ Très rapide (pas de connexion, pas d'attente d'ACK)
- ✓ Moins de surcharge réseau
- ✓ Idéal pour les applications temps réel

Inconvénients

- × Aucune garantie de livraison
- × Pas de réordonnancement ni de retransmission automatique
- × Risque de perte de paquets

Cas d'utilisation courants du protocole UDP :

- ✓ **Streaming** audio/vidéo (YouTube, Netflix en temps réel)
- ✓ **Jeux** en ligne
- ✓ **Appels** VoIP (Skype, Zoom)
- ✓ **DNS** (Domain Name System)
- ✓ **Diffusion** broadcast ou multicast sur un réseau (viséoconférence, …)